Wazuh API and Ansible API

From Detection Data to Controlled Remediation

Enterprise Vulnerability Remediation Platform

May 4, 2026

Roadmap

- Project context and global data flow
- How Wazuh scanning is performed
- How the Wazuh API works
- Important Wazuh API fields
- How the Ansible/AWX API works
- How both APIs are exploited by our platform
- Security controls and implementation path

Enterprise Platform: Formal Setup

Let A be the set of enterprise assets and V the set of detected vulnerabilities.

$$\text{A vulnerability record is linked to an asset:} \quad (\mathrm{a},\, \mathrm{v}) \in \mathrm{A} \times \mathrm{V}.$$

Our platform must transform raw security data into controlled remediation decisions.

$$\text{Detection} \rightarrow \text{Decision} \rightarrow \text{Execution} \rightarrow \text{Verification}$$

The objective is not only to detect weaknesses, but to build a governed process that can correct them safely.

Global Architecture

The platform is built around two main API families:

- **Wazuh APIs** expose detection, agents, inventory, vulnerabilities, and alerts.
- **Ansible/AWX API** launches approved automation jobs for remediation.

```
Endpoints
  |  Wazuh agents collect logs and inventory
  v
Wazuh Server
  |  Manager analyzes, Indexer stores
  v
Backend Platform
  |  Reads Wazuh data, decides policy
  v
Ansible AWX API
  |  Executes approved playbooks
  v
Endpoints corrected, then Wazuh verifies
```

How Wazuh Performs Scanning

Wazuh vulnerability detection is **inventory-based**, not port-based.

- When an endpoint starts, the Wazuh agent starts automatically.
- **Syscollector** collects OS, packages, apps, services, processes, users, local ports, and hotfixes.
- **Logcollector** collects security and system logs.
- The agent sends data to the Wazuh Manager on the agent communication channel.
- The manager compares inventory with vulnerability intelligence feeds.

The API does not launch the scan; it exposes the results after collection, analysis, and storage.

Wazuh Data Pipeline

1. Syscollector collects inventory
   OS, packages, versions, applications, hotfixes

2. Logcollector collects logs
   authentication, system events, application events

3. Wazuh agent sends data
   Agent -> Wazuh Manager, usually port 1514

4. Wazuh Manager analyzes data
   decoders, rules, vulnerability detection

5. Wazuh Indexer stores results
   alerts, inventory, vulnerability states

6. API exposes data
   backend retrieves useful fields

The ingestion path and the API access path are different. Agents send to the manager; our application reads from API and indexer.

Wazuh API: Two Access Layers

Layer	Role	Examples
Wazuh Server API	Manager and agent operations	`/agents`, `/manager/info`, `/syscollector/{id}/packages`
Wazuh Indexer API	Search stored security documents	`wazuh-alerts-*`, `wazuh-states-vulnerabilities-*`

The server API is convenient for agents and inventory. The indexer API is stronger for advanced search, vulnerability states, historical alerts, and analytics.

$$\text{Server API} = \text{operational access, Indexer API} = \text{searchable security data.}$$

Wazuh API Authentication

The backend authenticates to Wazuh, receives a JWT token, then uses it in later requests.

```
TOKEN=$(curl -k -u wazuh:'PASSWORD' \
  -X POST "https://WAZUH:55000/security/user/authenticate?raw=true")

curl -k -X GET "https://WAZUH:55000/agents?pretty=true" \
  -H "Authorization: Bearer $TOKEN"
```

- **POST** authenticates the API user.
- **JWT token** proves identity for later requests.
- **Bearer header** carries the token securely.
- In production, use a dedicated Wazuh API user with limited permissions.

Anatomy of a Wazuh API Request

Part	Example	Meaning
Method	`GET`, `POST`, `PUT`	Defines the operation: read, authenticate, modify, trigger.
URL	`https://server:55000/agents`	Defines the resource we want to access.
Header	`Authorization: Bearer token`	Proves the backend is authenticated.
Query params	`?pretty=true&limit=50&select=id,name,status`	Controls formatting, pagination, and selected fields.

Our backend should use `select`, filtering, limits, and pagination to avoid loading unnecessary data.

Wazuh API Response Structure

Most Wazuh Server API responses follow a common structure.

```
{
  "data": {
    "affected_items": [],
    "total_affected_items": 0,
    "total_failed_items": 0,
    "failed_items": []
  },
  "message": "...",
  "error": 0
}
```

- `error = 0` means success.
- `affected_items` contains the useful returned objects.
- `total_affected_items` helps with pagination and statistics.
- `failed_items` helps diagnose partial failures.

Important Wazuh Fields

Object	Fields	How we exploit them
Agent	`id, name, ip, status, lastKeepAlive, os.name, os.version`	Build assets page, scope remediation, check reachability.
Package	`name, version, architecture, vendor, format, scan.time`	Know what is installed and verify version after correction.
Hotfix	`hotfix, scan.time, agent_id`	Understand Windows patch state and missing KB updates.
Vulnerability	`vulnerability.id, severity, score, status, detected_at`	Prioritize CVEs and decide remediation flow.

Different machines can expose different fields. The backend normalizes them into stable internal objects.

Exploiting Wazuh API in Our Backend

The backend transforms raw Wazuh JSON into business objects for the enterprise platform.

```
GET /agents
  -> Asset(id, name, ip, os, status)

GET /syscollector/{id}/packages
  -> Package(agent_id, name, version, scan_time)

Search wazuh-states-vulnerabilities-*
  -> Vulnerability(agent_id, cve_id, severity, package, status)

POST /remediation-jobs
  -> controlled remediation workflow
```

The frontend never talks directly to Wazuh. It calls our backend, and our backend handles tokens, RBAC, normalization, and errors.

Why We Need Ansible API

Wazuh exposes vulnerabilities, but it does not provide a native API like:

POST /vulnerabilities/CVE-XXXX/fix

So remediation must be orchestrated by our platform.

- **Wazuh** detects and verifies.
- **Backend** decides whether correction is allowed.
- **Ansible/AWX** executes approved correction playbooks.
- **PostgreSQL** stores jobs, approvals, and history.

Ansible is the execution engine, not the detection engine.

Ansible and AWX API

Ansible automates tasks using playbooks. AWX or Ansible Automation Platform exposes those playbooks through a REST API.

Concept	Meaning	Use in our project
Inventory	List of managed machines	Linux/Windows targets grouped by environment
Playbook	Automation file written in YAML	Upgrade package, restart service, verify version
Job template	Approved executable playbook configuration	The only unit our backend is allowed to launch
Job	One execution of a template	Tracked in database and audit logs

AWX API Authentication

The backend authenticates to AWX using a token or configured credentials.

```
curl -X POST "https://AWX/api/v2/job_templates/12/launch/" \
  -H "Authorization: Bearer AWX_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "extra_vars": {
      "target_agent": "debian-agent",
      "package_name": "openssl"
    }
  }'
```

- The token belongs to a service account, not a human user.
- The service account can launch only approved job templates.
- Sensitive credentials stay in AWX or a vault, not in frontend code.

Launching a Remediation Job

The main AWX endpoint for execution is the job template launch endpoint.

$$\text{POST } /\text{api/v2/job_templates/\{id\}/launch/}$$

Input	Description	Security rule
`job_template_id`	Which approved playbook is executed	Chosen by backend policy, not by user text
`inventory`	Which hosts can be targeted	Restricted by environment and asset scope
`extra_vars`	Variables such as package name or service name	Validated and whitelisted by backend
`credentials`	SSH/WinRM access	Stored in AWX/Vault, never in frontend

Example Playbook Logic

A Linux package remediation playbook can be controlled and reusable.

```
---
- name: Upgrade vulnerable package
  hosts: "{{ target_host }}"
  become: true
  tasks:
    - name: Refresh apt cache
      apt:
        update_cache: yes

    - name: Upgrade selected package only
      apt:
        name: "{{ package_name }}"
        state: latest
```

The backend must validate `target_host` and `package_name` before launching the job.

Complete Exploitation Flow

1. Backend authenticates to Wazuh
2. Backend retrieves active vulnerabilities
3. Backend normalizes fields:
   agent, OS, package, version, CVE, severity
4. Backend checks remediation policy
5. Backend checks RBAC and approval state
6. Backend calls AWX API:
   POST /job_templates/{id}/launch/
7. AWX runs the playbook on target endpoint
8. Backend stores stdout, stderr, exit code
9. Wazuh agent rescans inventory
10. Backend checks Wazuh again:
    Active -> failed / pending
    Solved -> resolved

Database Role in API Exploitation

The application database is not a replacement for Wazuh. It stores our business state.

Table	Data stored	Why it matters
`assets`	Normalized Wazuh agents	Stable asset inventory for frontend and policies
`vulnerabilities`	Normalized CVE records	Prioritization and remediation tracking
`remediation_jobs`	AWX job mapping and status	Trace every correction attempt
`audit_logs`	User, role, action, target, time	Enterprise accountability

Security Controls

Risk	Control	Implementation
Exposed Wazuh credentials	Backend-only access	Frontend never calls Wazuh directly
Unauthorized remediation	RBAC and approvals	Viewer, analyst, approver, admin roles
Command injection	No free commands	Only approved playbooks and whitelisted variables
False success	Post-correction verification	Wazuh must confirm vulnerability is solved
Lost accountability	Audit logs	Record user, action, target, output, result

Implementation Order

- Build FastAPI backend foundation and `GET /health`.
- Create Wazuh API client and return normalized `/assets`.
- Add vulnerability ingestion from Wazuh/indexer.
- Create PostgreSQL models for assets, vulnerabilities, jobs, audit logs.
- Add Ansible/AWX API client and a first safe job template.
- Add approval workflow and post-remediation verification.
- Build frontend dashboard after the backend data model is stable.

Conclusion

Wazuh API gives us detection data.

Ansible API gives us controlled execution.

Our backend connects both through policy, database, audit, and verification.

Wazuh detects → Backend decides → Ansible corrects → Wazuh verifies